

Tugas 2

Nama: Muhammad Farisi Suyitno

NIM: 123140152

<https://github.com/lcniP/stm-if25-40305-123140152>

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
import librosa #untuk menampilkan visualisasi data audio
import librosa.display
from scipy.fft import fft, fftfreq #fungsi untuk mengubah sinyal suara menjadi frek
import warnings

warnings.filterwarnings('ignore')
```

```
In [3]: # Memuat file audio dengan sample rate asli
file_path = 'Berita.wav'
y, sr = librosa.load(file_path, sr=None)

# Menghitung metrik dasar
durasi = librosa.get_duration(y=y, sr=sr)
jumlah_sampel = len(y)
min_amp = np.min(y)
max_amp = np.max(y)

# Mencetak hasil inspeksi awal
print("=== INSPEKSI AWAL AUDIO ===")
print(f"Laju Sampel Asli (fs) : {sr} Hz")
print(f"Durasi Audio : {durasi:.2f} detik")
print(f"Jumlah Total Sampel : {jumlah_sampel} sampel")
print(f"Amplitudo Minimum : {min_amp:.5f}")
print(f"Amplitudo Maksimum : {max_amp:.5f}")
```

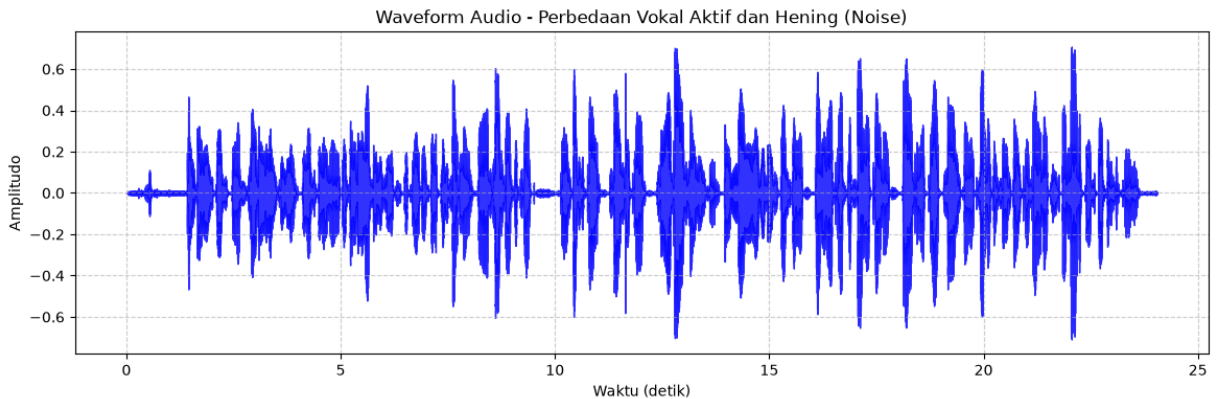
```
=== INSPEKSI AWAL AUDIO ===
Laju Sampel Asli (fs) : 48000 Hz
Durasi Audio : 24.04 detik
Jumlah Total Sampel : 1154048 sampel
Amplitudo Minimum : -0.60428
Amplitudo Maksimum : 0.70892
```

Visualisasi Audio 4 Dimensi

Waveform (Amplitudo vs Waktu) Grafik di bawah ini merepresentasikan sinyal audio dalam domain waktu. Amplitudo (sumbu-Y) menunjukkan seberapa kuat tekanan suara pada waktu tertentu (sumbu-X).

```
In [4]: # Membuat plot Waveform
plt.figure(figsize=(14, 4))
```

```
librosa.display.waveshow(y, sr=sr, alpha=0.8, color='blue')
plt.title('Waveform Audio - Perbedaan Vokal Aktif dan Hening (Noise)')
plt.xlabel('Waktu (detik)')
plt.ylabel('Amplitudo')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



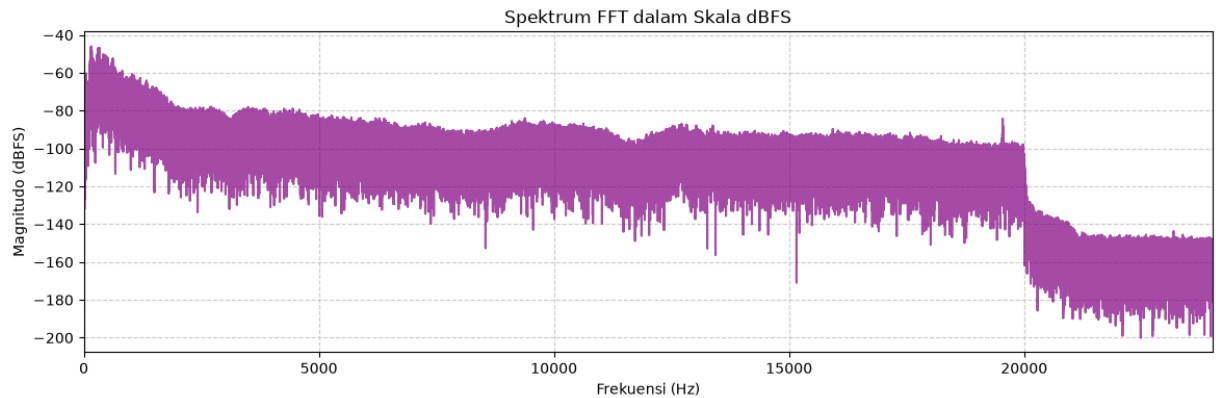
Spektrum FFT dalam Skala dBFS (Full Scale)

Fast Fourier Transform (FFT) memecah gelombang kompleks menjadi komponen-komponen frekuensi dasarnya. Untuk merepresentasikan kenyaringan seperti yang didengar telinga, magnitudo diubah ke skala desibel (dBFS) menggunakan rumus $20 \log_{10}(\text{Magnitudo})$.

```
In [5]: # Menghitung jumlah sampel dan frekuensi sumbu X
N = len(y) #mengambil sampel titik data audio
yf = fft(y) #mengubah sinyal audio menjadi domain frekuensi
xf = fftfreq(N, 1/sr)[:N//2] #mengambil frekuensi positif (sumbu X) hingga frekuensi

# Menghitung magnitudo dan mengubah ke skala dBFS
# Ditambahkan epsilon (1e-10) untuk menghindari error Log10(0)
yf_mag = 2.0/N * np.abs(yf[:N//2])
yf_dbfs = 20 * np.log10(np.maximum(1e-10, yf_mag))

# Membuat plot FFT
plt.figure(figsize=(14, 4))
plt.plot(xf, yf_dbfs, color='purple', alpha=0.7)
plt.title('Spektrum FFT dalam Skala dBFS')
plt.xlabel('Frekuensi (Hz)')
plt.ylabel('Magnitudo (dBFS)')
plt.xlim(0, sr/2) # Membatasi ke frekuensi Nyquist
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

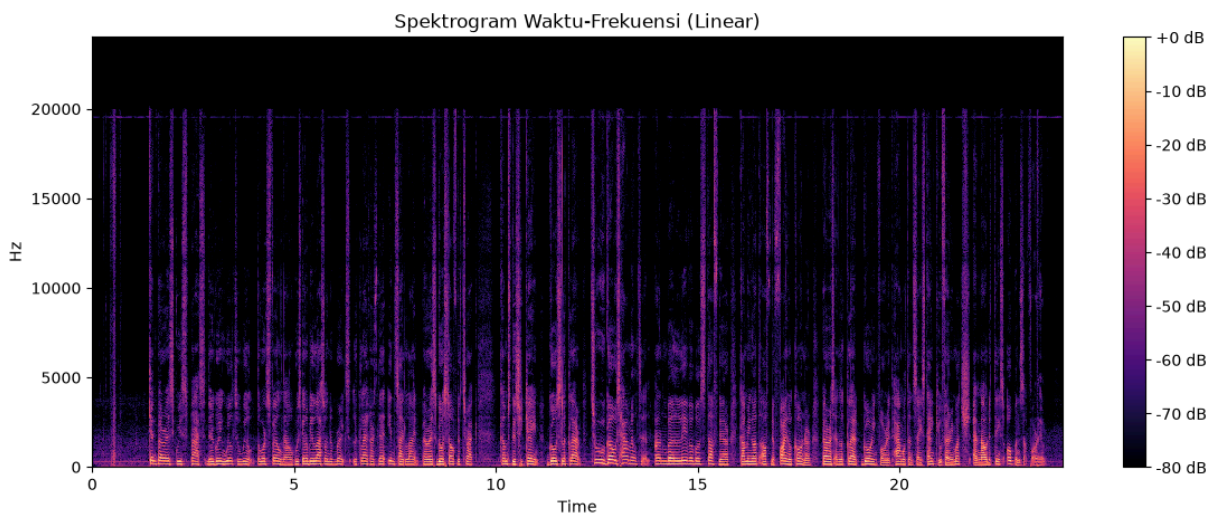


Spektrogram (STFT)

Short-Time Fourier Transform (STFT) memberikan pandangan 3 dimensi: waktu (X), frekuensi (Y), dan intensitas/energi (Warna/Z).

```
In [6]: # Menghitung STFT dan mengubah amplitudo ke desibel
D = librosa.stft(y)
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)

# Membuat plot Spektrogram
plt.figure(figsize=(14, 5))
librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='hz', cmap='magma')
plt.colorbar(format='%+2.0f dB')
plt.title('Spektrogram Waktu-Frekuensi (Linear)')
plt.show()
```

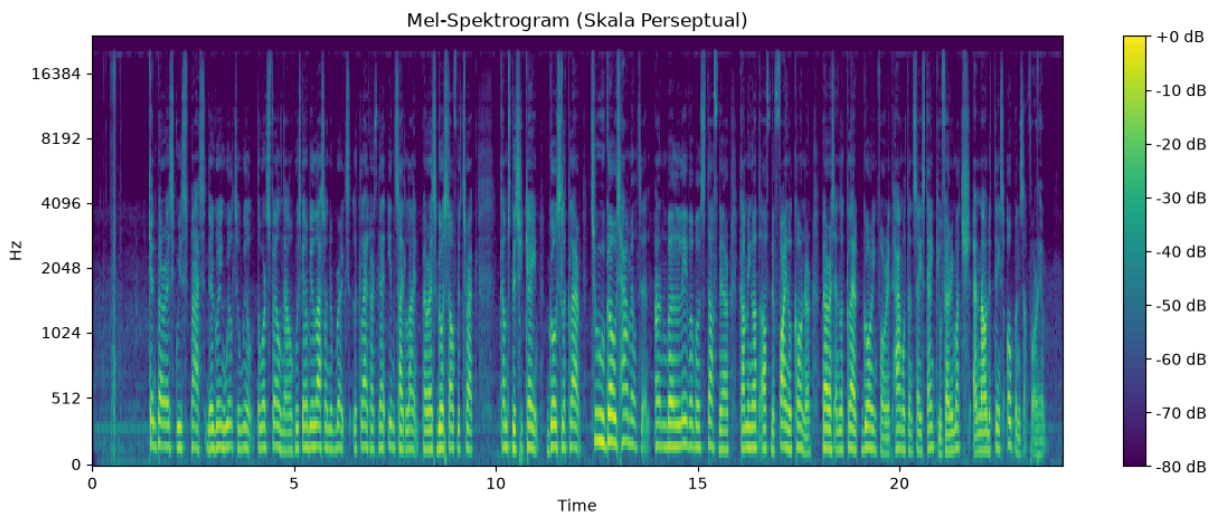


Mel-Spektrogram

Mel-Spektrogram mengubah skala linier frekuensi (Hz) menjadi skala Mel, yang meniru persepsi pendengaran telinga manusia secara logaritmik (lebih sensitif pada frekuensi rendah, kurang sensitif pada frekuensi tinggi).

```
In [7]: # Mengekstrak fitur mel-spectrogram
M = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128)
M_db = librosa.power_to_db(M, ref=np.max)

# Membuat plot Mel-Spektrogram
plt.figure(figsize=(14, 5))
librosa.display.specshow(M_db, sr=sr, x_axis='time', y_axis='mel', cmap='viridis')
plt.colorbar(format='%+2.0f dB')
plt.title('Mel-Spektrogram (Skala Perseptual)')
plt.show()
```



Eksperimen Resampling & Pembuktian Aliasing

Pada bagian ini, kita melakukan eksperimen downsampling dari laju sampel asli (biasanya 44.1 kHz atau 48 kHz) ke laju yang jauh lebih rendah (misal 8 kHz). Sesuai Teorema Nyquist, laju 8 kHz hanya mampu menampung frekuensi maksimum 4 kHz. Frekuensi di atas 4 kHz harus dibuang agar tidak terjadi tabrakan.

Kita membandingkan 2 skenario:

1. **Skenario A (Naive Decimation):** Mengambil 1 sampel setiap M loncatan, tanpa difilter terlebih dahulu.
2. **Skenario B (Resampling Standar):** Menggunakan fungsi `librosa.resample` yang secara otomatis menyertakan *Low Pass Filter* (LPF) anti-aliasing sebelum membuang sampel.

```
In [8]: # Menentukan laju sampel target dan faktor decimasi (M)
# Misal: Kita pangkas ke 8000 Hz
sr_target = 8000
M = int(sr / sr_target)

# Jika SR asli tidak persis kelipatan 8000, hitung sr_naif aktualnya
sr_naif = sr // M

# SKENARIO A: Naive Decimation (Mengambil tiap sampel ke-M langsung)
y_naif = y[::M]
```

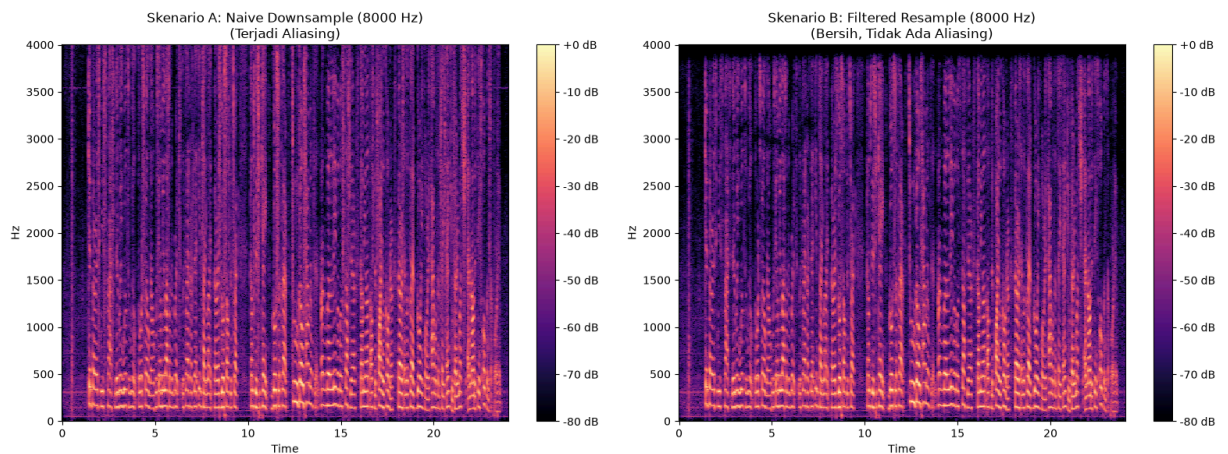
```
# SKENARIO B: Resampling Standar dengan Filter LPF (Anti-Aliasing)
# librosa.resample secara internal menggunakan scipy.signal.resample_poly
y_benar = librosa.resample(y, orig_sr=sr, target_sr=sr_target)

# Membuat visualisasi perbandingan spektrogram
plt.figure(figsize=(16, 6))

# Plot Spektrogram Skenario A (Naif)
plt.subplot(1, 2, 1)
D_naif = librosa.amplitude_to_db(np.abs(librosa.stft(y_naif)), ref=np.max)
librosa.display.specshow(D_naif, sr=sr_naif, x_axis='time', y_axis='hz', cmap='magma')
plt.title(f'Skenario A: Naive Downsample ({sr_naif} Hz)\n(Terjadi Aliasing)')
plt.colorbar(format='%+2.0f dB')

# Plot Spektrogram Skenario B (Resample Terfilter)
plt.subplot(1, 2, 2)
D_benar = librosa.amplitude_to_db(np.abs(librosa.stft(y_benar)), ref=np.max)
librosa.display.specshow(D_benar, sr=sr_target, x_axis='time', y_axis='hz', cmap='magma')
plt.title(f'Skenario B: Filtered Resample ({sr_target} Hz)\n(Bersih, Tidak Ada Aliasing)')
plt.colorbar(format='%+2.0f dB')

plt.tight_layout()
plt.show()
```



```
In [12]: import soundfile as sf

# Ganti 'audio_a' dan 'audio_b' dengan nama variabel aslimu yang ada di cell atas
sf.write('audio_downsampled_naive.wav', D_naif, 8000)
sf.write('audio_downsampled_clean.wav', D_benar, 8000)

print("File audio berhasil disimpan!")
```

File audio berhasil disimpan!

Kesimpulan Berdasarkan Hasil Kedua Skenario tersebut

dapat dilihat di bagian tengah ke atas 2000 sampai 4000 Hz. terlihat ada garis warna ungu dan pink. Itulah aliasing-nya.

Bandingkan dengan grafik Skenario B. Di grafik B, area atasnya berwarna hitam, yang berarti bersih dari noise. Sebaliknya di Skenario A, area penuh dengan garis warna.

Kenapa Bisa Terjadi? garis ungu di Skenario A muncul karena ukuran audio dipaksa kecil tanpa di-filter terlebih dahulu. Akibatnya, suara-suara bernada sangat tinggi yang seharusnya dibuang masuk ke bawah dan masuk ke rentang frekuensi yang terlihat di layar. Suara gangguan ini menumpuk dengan suara asli, sehingga kalau didengarkan akan seperti noise yang mengganggu.